

Implementation and Performance Analysis of SDN Traffic Forwarding and Redirection with Mininet and Ryu

Binyu Li

*School of Advanced Technology
Xi'an Jiaotong Liverpool University
Suzhou, China*

Binyu.Li23@student.xjtlu.edu.cn
Student ID: 2364567

Junyan He

*School of Advanced Technology
Xi'an Jiaotong Liverpool University
Suzhou, China*

Junyan.He23@student.xjtlu.edu.cn
Student ID: 2363304

Chenyu Du

*School of Advanced Technology
Xi'an Jiaotong Liverpool University
Suzhou, China*

Chenyu.Du23@student.xjtlu.edu.cn
Student ID: 2360743

Zengyi Mei

*School of Advanced Technology
Xi'an Jiaotong Liverpool University
Suzhou, China*

Zengyi.Mei23@student.xjtlu.edu.cn
Student ID: 2361796

Jiafei Sun

*School of Advanced Technology
Xi'an Jiaotong Liverpool University
Suzhou, China*

Jiafei.Sun23@student.xjtlu.edu.cn
Student ID: 2361510



CAN201 - GROUP 29

Advisors: Dr. Fei Cheng, Dr. Gordon Boateng

Abstract—This project builds an SDN testbed using Mininet and the Ryu controller to realize dynamic forwarding and TCP redirection through IP/MAC rewriting. Experimental results verify correct forwarding, redirection, and latency differences between the two modes.

Index Terms—SDN, Ryu controller, Mininet, OpenFlow, Flow redirection, TCP forwarding

I. INTRODUCTION

Traditional IP networks that tightly couple control logic with data forwarding and rely on static hardware configurations are increasingly difficult to operate and scale in modern enterprise environments. Their limited flexibility makes it challenging to respond quickly to failures, shifting traffic patterns, or new service requirements. Software-Defined Networking (SDN) addresses these limitations by decoupling the control and data planes, enabling centralized, programmable traffic management through interfaces such as OpenFlow. This architecture provides fine-grained control over forwarding behaviour and forms the foundation for flexible mechanisms such as dynamic load balancing, traffic steering, and enhanced network security. In this project, we build an SDN-based testbed using Mininet and a Ryu controller to examine how controller logic can transparently influence application traffic without modifying client behaviour.

A. Project Objective

The goal of this project is to construct a simple SDN topology in Mininet and implement controller-driven traffic control at the flow level. The topology includes one client and two servers connected through a single OpenFlow switch managed by a Ryu controller. Although the client is configured to communicate only with Server1, the controller dynamically determines whether flows are forwarded normally or transparently redirected to Server2 by installing appropriate flow entries.

B. Challenges

The primary challenge is to design controller applications that provide dynamic, per-flow control while maintaining correct network operation. The controller must preserve basic host reachability using short-lived flow rules, accurately detect the initial SYN packet of each TCP connection, and install bidirectional flow entries with a 5-second idle timeout without disrupting the TCP three-way handshake. For traffic redirection, the controller must also rewrite IP and MAC headers consistently in both directions, ensuring that the client perceives an unchanged communication endpoint even when the actual session terminates at a different server.

C. Practical Relevance

Despite its small scale, the implemented testbed reflects real-world SDN use cases. Controller-driven flow steering underpins application-layer load balancing, allowing connections to be distributed across multiple servers without altering client settings. Transparent traffic redirection and header rewriting also support security applications, such as directing suspicious flows to inspection modules. Furthermore, short-lived per-flow rules enable precise utilization of switch resources and rapid adaptation to dynamic traffic conditions in enterprise or data-centre networks.

D. Contributions

In this coursework project, we accomplished the following key tasks:

1) Constructed a Mininet-based SDN topology consisting of one client, two servers, and an OpenFlow switch, with all IP and MAC addresses configured according to the specification and connected to a remote SDN controller.

2) Developed SDN controller applications using the Ryu framework to provide dynamic per-flow control. The controller maintains host reachability, installs short-lived bidirectional flow entries on the first TCP SYN with a 5-second idle timeout, and supports transparent traffic redirection through consistent IP/MAC header rewriting.

3) Implemented and executed TCP client and server programs to generate application-level traffic, validate normal forwarding and redirection behaviour, and analyze TCP handshake latency using packet-capture tools such as Wireshark.

II. RELATED WORK

Network traffic redirection is important in modern network management, enabling load balancing and resource allocation. SDN and NFV are also used in network slicing to enable flexible resource allocation [1]. In the domain of cybersecurity, Hyder et al. [2] show that SDN-based Moving Target Defense (MTD) systems can redirect malicious traffic to shadow hosts, preventing attackers from reaching real services during Crossfire-style DDoS attacks. Moreover, OpenFlow offers the programmability required to define customized forwarding behaviors within SDN environments [2], [3]. Wang et al. [3] further highlight that SDN/OpenFlow architectures enable precise tracing of forwarding paths, providing an effective method for diagnosing data-plane states and validating flow-table configurations.

III. DESIGN

A. Architecture

The system adopts an SDN architecture with one switch and three hosts, managed by a remote Ryu controller using OpenFlow 1.3. Switch *s1* links the client and two servers, each with fixed IP and MAC addresses for consistent flow matching, as shown in Fig. 1.

When the switch first connects, the controller installs a table-miss rule and builds a *mac_to_port* mapping from Packet-In events. During normal forwarding, once a destination MAC is known, the controller installs short-lived bidirectional flow entries for ICMP packets and for the first TCP SYN of each new connection.

In redirect mode, the physical topology remains unchanged, but forwarding behavior is modified. When the client initiates a TCP connection to *server1*, the controller intercepts the first SYN and installs a pair of rewrite rules that transparently redirect the connection to *server2*.

- **Forward rule:** Rewrite the destination MAC and IP to those of *server2* and output the packet on its corresponding port.
- **Reverse rule:** Rewrite response packets from *server2* to use *server1*'s MAC and IP, preserving transparency.

Both redirect rules use the same five-second idle timeout.

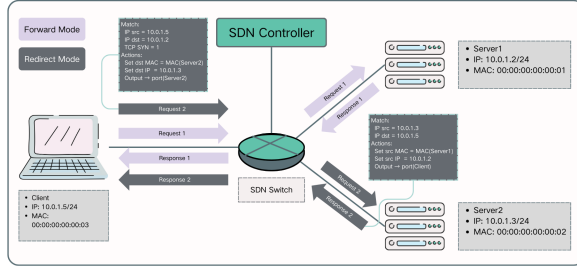


Fig. 1: System topology consisting of a client, two servers.

B. Workflow

The system workflow is driven by Packet-In events and the installation of flow entries. When a switch first connects, the controller installs a table-miss rule to ensure that the first packet of every new flow is sent to the controller for inspection.

1) *Forwarding workflow*: Upon receiving a Packet-In, the controller parses the Ethernet header, learns the source MAC address, and updates the MAC-port table. As shown in Fig. 2a, if the destination MAC is known, the packet is forwarded to the correct port; otherwise, it is flooded.

For ICMP packets, the forward rule matches the incoming port and IP fields, and the reverse rule matches the opposite direction. The forward entry uses the Packet-In `buffer_id` so the triggering packet is forwarded immediately.

For TCP packets, flows are installed only when the first SYN is observed and the output port is known. A bidirectional pair is created using IP addresses and TCP ports, with the forward rule again using the SYN packet's `buffer_id`, while the reverse rule is installed normally. All subsequent packets of the connection are handled directly by these flow entries.

2) *Redirection workflow*: In redirection mode, the controller still performs MAC learning and flooding, but additionally inspects TCP SYN packets. As shown in Fig. 2b, when the client sends a SYN to *server1* and the port of *server2* is known, two high-priority flow entries are created to transparently redirect the connection.

Flow entry creation:

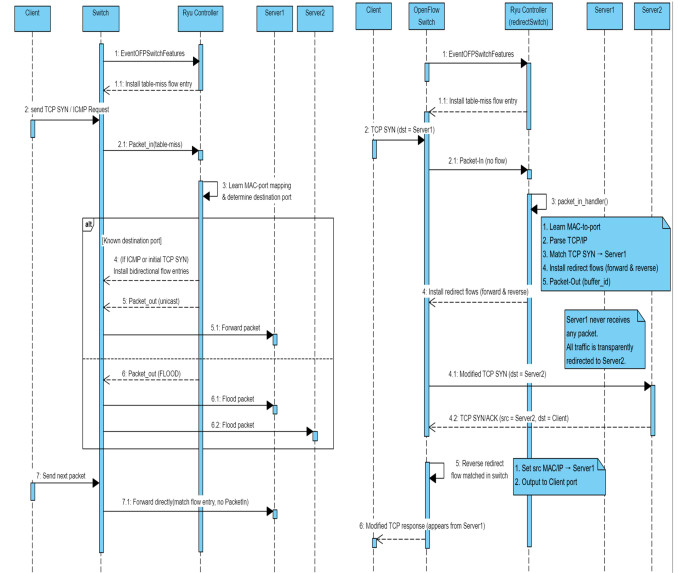
- The forward rule matches client-to-*server1* TCP traffic (SYN packets).
- The reverse rule matches *server2*-to-client response traffic.

Flow entry installation:

- The forward rule rewrites the destination IP and MAC addresses to those of *server2* and outputs the packet on its port using the Packet-In `buffer_id`.
- The reverse rule rewrites the source IP and MAC addresses to those of *server1* before sending packets back to the client.

C. Algorithm

The forwarding controller operates as a learning switch, installing ICMP or TCP flow entries only when required.



(a) Forward sequence

(b) Redirection sequence

Fig. 2: Sequence diagrams for forwarding and TCP redirection workflows.

The redirection controller adds one additional step: It captures the client's initial TCP SYN and sets bidirectional rules for transparent redirection to Server2. The forwarding process is summarized in Algorithm 1, and the redirection process is shown in Algorithm 2.

IV. IMPLEMENTATION

A. Development Environment

The environment of development is shown in Table I.

TABLE I: Development Environment

Field	Description
Operating System	Microsoft Windows 11
CPU	12th Gen Intel(R) Core(TM) i7-12700H
RAM	16 GB
IDE	JetBrains PyCharm Professional 2022.2.4
Python Version	Python 3.10
Python Library	'time', 'socket', 'mininet', 'ryu',
SDN Controller	Ryu 4.34

B. Implementation Steps

As shown in Fig. 3, we followed a systematic approach to implement the forwarding and redirection functionality.

1) Reviewed the coursework requirements, extracted all essential tasks, and created activity/sequence diagrams to model the interaction flow.

2) Studied the operational principles of the Ryu controller, focusing on its event-driven workflow and OpenFlow message handling.

3) Analyzed and refactored the provided example code to improve structure, naming clarity, and overall readability.

4) Implemented the forwarding logic in Ryu as a learning switch, including MAC learning, installation of short-lived

Algorithm 1 Forwarding controller workflow

```
1: On switch connection:
2: Install table-miss flow: match all; action  $\leftarrow$  OUTPUT(CONTROLLER); idle_timeout = 0.
3:
4: On Packet-In from switch:
5: Parse Ethernet header eth.
6: if eth.ethertype is LLDP then
7:   Drop packet and return.
8: end if
9: Learn source MAC and input port:
   mac_to_port[dpid][src]  $\leftarrow$  in_port.
10: if destination MAC is known then
11:   out_port  $\leftarrow$  mac_to_port[dpid][dst].
12: else
13:   out_port  $\leftarrow$  FLOOD.
14: end if
15: Parse IPv4 header ip_pkt (if any).
16: if ip_pkt exists and ip_pkt.proto is ICMP and
   out_port  $\neq$  FLOOD then
17:   Install short-lived bidirectional ICMP flows between
   (srcIP, dstIP) with priority 10, idle_timeout = 5;
   forward flow uses buffer_id.
18: else if ip_pkt exists and ip_pkt.proto is TCP then
19:   Parse TCP header tcp_pkt, read SYN and ACK flags.

20:   if SYN = 1, ACK = 0 and out_port  $\neq$  FLOOD
   then
21:     Install short-lived bidirectional TCP flows for the 4-
     tuple (srcIP, srcPort, dstIP, dstPort) with priority
     20, idle_timeout = 5; forward flow uses buffer_id.
22:   end if
23: end if
24: Send PacketOut with OUTPUT(out_port) using buffer_id
   (if available) or raw data.
```

bidirectional flow entries for ICMP traffic and for TCP connections triggered by the first SYN segment, and detailed TCP packet handling.

5) Developed the redirection mechanism in the controller by rewriting Ethernet and IPv4 headers using `OFActionSetField` to transparently redirect traffic from the logical Server1 address to Server2 while keeping the client's view unchanged.

6) Deployed the Mininet topology and Ryu controllers in a virtualized environment and performed end-to-end tests with the provided TCP client and server applications to verify forwarding and redirection behaviour.

7) Added detailed comments and applied minor optimizations to enhance modularity, maintainability, and runtime efficiency.

8) Collected test data, analyzed the results, and summarized the findings in the final report.

C. Programming Skills

- **Abstraction and Encapsulation:** Repetitive or complex operations—such as constructing match-action rules, generating flow entries, and assembling modified packet

Algorithm 2 Redirection controller workflow

```
1: On switch connection:
2: Install table-miss flow: match all; action  $\leftarrow$  OUTPUT(CONTROLLER); idle_timeout = 0.
3:
4: On Packet-In from switch:
5: Parse Ethernet header eth.
6: if eth.ethertype is LLDP then
7:   Drop packet and return.
8: end if
9: Learn source MAC and input port:
   mac_to_port[dpid][src]  $\leftarrow$  in_port.
10: if destination MAC is known then
11:   out_port  $\leftarrow$  mac_to_port[dpid][dst].
12: else
13:   out_port  $\leftarrow$  FLOOD.
14: end if
15: actions  $\leftarrow$  OUTPUT(out_port).
16: Parse IPv4 header ip_pkt.
17: if ip_pkt exists and ip_pkt.proto is TCP then
18:   Parse TCP header tcp_pkt and check flags.
19:   if SYN = 1, ACK = 0, srcIP = CLIENT_IP,
   dstIP = SERVER1_IP then
20:     Look up server2_port using SERVER2_MAC.
21:     if server2_port is known then
22:       Install high-priority bidirectional redirect flows
       (priority 50, idle_timeout = 5):
23:       Forward: match client  $\rightarrow$  Server1 (IP + TCP 4-
       tuple), rewrite dst MAC/IP to SERVER2_MAC,
       SERVER2_IP and OUTPUT(server2_port),
       using buffer_id.
24:       Reverse: match Server2  $\rightarrow$  client (4-
       tuple swapped), rewrite src MAC/IP to
       SERVER1_MAC, SERVER1_IP and
       OUTPUT(in_port), without buffer_id.
25:       actions  $\leftarrow$  None {current SYN is handled by the
       new redirect flow}
26:     end if
27:   end if
28: end if
29: if actions  $\neq$  None then
30:   Send PacketOut with OUTPUT(out_port) using
   buffer_id (if available) or raw data.
31: end if
```

headers—are encapsulated into helper functions. This abstraction reduces duplication, streamlines the workflow, and creates a clearer separation between high-level logic and low-level implementation.

- **Object-Oriented Programming (OOP):** The Ryu application is built using OOP principles by extending the base class `app_manager.RyuApp`. Through inheritance and method overriding, the controller integrates custom behaviours while maintaining compatibility with the Ryu event system, ensuring both extensibility and structural organization.
- **Event-Driven Programming:** Leveraging Ryu's event-driven architecture, the controller reacts to OpenFlow

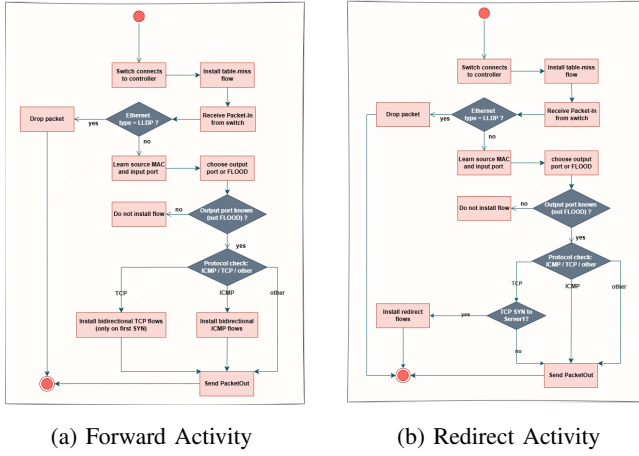


Fig. 3: Activity diagram of forward and redirect.

messages such as PacketIn and SwitchFeatures. This approach improves responsiveness to network state changes and supports dynamic decision-making during forwarding and redirection.

- **High Cohesion and Low Coupling:** Related functionalities are grouped within cohesive modules, while interactions between these modules are kept minimal. This design choice enhances flexibility, reduces side-effects when changes are made, and supports cleaner integration during incremental development.
- **Network Packet Manipulation Skills:** The implementation involves reading, interpreting, and modifying various protocol headers (Ethernet, IPv4, TCP). This strengthens low-level network programming capabilities and ensures accurate processing during traffic forwarding and redirection.

D. Actual Implementation:

- **SDN Network Topology:** The topology is implemented using the Mininet Python API. A remote controller *c0* is configured, and three hosts (Client, Server1, Server2) are connected to a single Open vSwitch *s1* via *addLink*. Each host is assigned the IP and MAC addresses specified in the coursework (Client: 10.0.1.5/24, Server1: 10.0.1.2/24, Server2: 10.0.1.3/24), ensuring that the emulated network matches the project specification. Once the topology script is executed, the network is started and inspected via the Mininet CLI.
- **Forwarding:** The client and Server1 applications generate TCP traffic on their respective hosts. When the client sends an initial TCP SYN to Server1 and no matching flow exists, switch *s1* sends a PacketIn to the controller, which parses the Ethernet/IP/TCP headers, learns MAC–port mappings and, if necessary, temporarily floods the packet. Once the ports of the client and Server1 are known, the controller installs short-lived bidirectional TCP flow entries (5 s idle timeout) on *s1* that match the connection 4-tuple and forward packets to the correct ports. Subsequent handshake and data packets are then handled entirely in the data plane without further controller involvement.

- **Redirecting:** In the redirection scenario, the client still targets Server1 while the server application actually runs on Server2. A SYN from the client to Server1 triggers a PacketIn; the controller detects this flow, rewrites the destination MAC and IP to those of Server2 using *OFActionSetField*, outputs the packet on Server2’s port, and installs a forward-direction redirect rule with a short idle timeout. When Server2 replies with a SYN/ACK, the controller similarly rewrites the source MAC and IP from Server2 to Server1, installs the reverse redirect rule, and forwards the packet to the client. Afterwards, all packets are transparently translated by these bidirectional rules, so the client believes it is communicating with Server1 even though the session terminates at Server2.

- **Difficulties and Solutions:** The main difficulty was implementing and debugging the TCP redirection logic. Initially, only the output port was changed, while the Ethernet and IP headers were left unchanged, causing Server2 to reject the redirected TCP segments and the three-way handshake to fail. By combining Ryu’s logging facilities with Wireshark/tcpdump traces, this header mismatch was identified and the controller was updated to consistently rewrite both MAC and IP fields in both directions and to install the corresponding redirect flows on the switch. This resolved the connectivity issues and resulted in stable TCP redirection.

V. TESTING AND RESULTS

A. Testing Environment

The testing environment is shown in Table II.

TABLE II: System Configuration

Field	Description
Operating System	Ubuntu 20.04.6
CPU	13th Gen Intel(R) Core(TM) i9-13900HX
RAM	16 GB
IDE	JetBrains PyCharm Professional 2024.1.3
Python Version	Python 3.8.10
Python Library	socket, time, mininet, ryu
SDN controller	Ryu 4.34

B. Test Step

In the following, we perform a stepwise usability evaluation of the SDN network topology instantiated using the Mininet Python library.

1) *Step 1. Network Topology and Address Verification:* We first verified that the Mininet topology matched the coursework specification. Execute `networkTopo.py`, and then use the `net` command to verify that the controller *c0*, switch *s1*, and the three hosts are correctly interconnected. Subsequently, use `ifconfig` on the client and server hosts to confirm that their IP and MAC address assignments are accurate, as illustrated in Fig. 4. Thus, the topology and address configuration are validated as correct for the subsequent experiments.

2) *Step 2. Connectivity and Flow Entry Verification:* When the Mininet topology was running, the SDN controller was started on node *c0* by executing `ryu_forward.py` or `ryu_redirect.py`, and the establishment of the

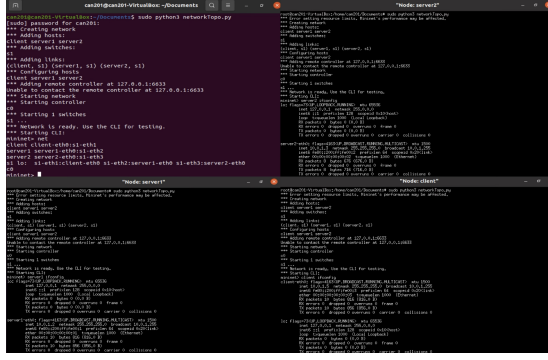
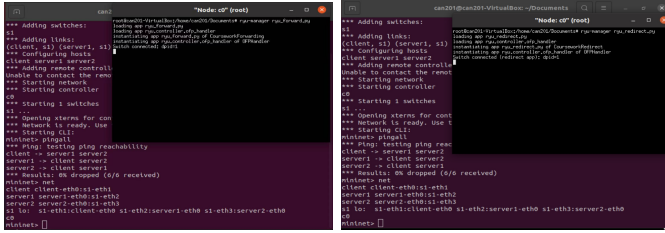


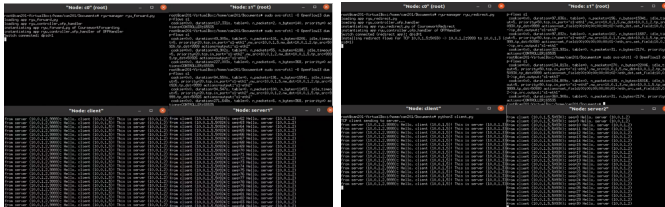
Fig. 4: IP and MAC Address Validation

OpenFlow session with switch *s1* was confirmed. The `pingall` command was then used to test end-to-end reachability among all internal nodes, as shown in Fig. 5. Finally, `sudo ovs-ofctl -O OpenFlow13 dump-flows s1` was executed on the switch; the resulting flow table contained ICMP and TCP entries with `idle_timeout` set to 5, confirming that all flow entries were configured with an idle timeout of five seconds, as illustrated in Fig. 6.



(a) Network Topology and Ping (b) Network Topology and Ping are both executed when the controller performs forwarding

Fig. 5: Network Topology and Pingall Process

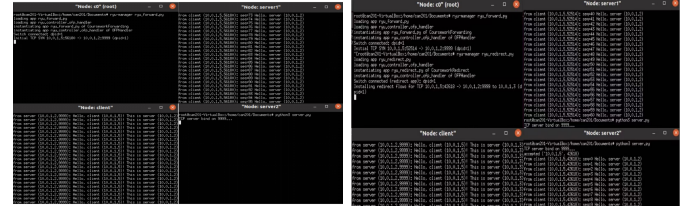


(a) Flow entries in forwarding deployment (b) Flow entries in redirecting deployment

Fig. 6: Flow Entries of Switch.

3) *Step 3. Traffic Forwarding:* The `ryu_forward.py` application was run on controller node *c0* to enable the forwarding logic in Fig. 7a. The provided `server.py` was subsequently started on *server1* and *server2*, followed by running `client.py` on the client to generate TCP traffic toward *server1*. After the TCP connection was established, the `pingall` command was re-issued to confirm full IP connectivity across all nodes. Concurrently, Wireshark was

configured on the client to capture interface packets, the trace in Fig. 8 confirms correct traffic forwarding to *server1* under the forwarding implementation.



(a) Traffic Forwarding

(b) Traffic Redirection

Fig. 7: Forward and Redirect Implementation process

Time	Source	Destination	Protocol	Length	Info
1.0.000000000	10.0.1.5	10.0.1.2	TCP	96	56186 → 9999 [PSH, ACK] Seq=1414141414 Win=65535 Len=0
2.0.00013461	10.0.1.2	10.0.1.5	TCP	117	9999 → 56186 [PSH, ACK] Seq=1414141414 Win=65535 Len=0
3.0.000120922	10.0.1.5	10.0.1.2	TCP	96	56186 → 9999 [ACK] Seq=1414141414 Win=65535 Len=0
4.1.0001902031	10.0.1.5	10.0.1.2	TCP	96	56186 → 9999 [PSH, ACK] Seq=3141414141 Win=65535 Len=0
5.1.0002028769	10.0.1.2	10.0.1.5	TCP	117	9999 → 56186 [PSH, ACK] Seq=5214141414 Win=65535 Len=0
6.1.00020306197	10.0.1.5	10.0.1.2	TCP	96	56186 → 9999 [ACK] Seq=6141414141 Win=65535 Len=0
7.2.000319245	10.0.1.5	10.0.1.2	TCP	96	56186 → 9999 [PSH, ACK] Seq=6141414141 Win=65535 Len=0
8.2.0003230864	10.0.1.2	10.0.1.5	TCP	117	9999 → 56186 [PSH, ACK] Seq=10314141414 Win=65535 Len=0
9.2.0003238278	10.0.1.5	10.0.1.2	TCP	96	56186 → 9999 [ACK] Seq=9141414141 Win=65535 Len=0
10.3.0004951128	10.0.1.5	10.0.1.2	TCP	96	56186 → 9999 [PSH, ACK] Seq=9141414141 Win=65535 Len=0
11.3.000510249	10.0.1.2	10.0.1.5	TCP	117	9999 → 56186 [PSH, ACK] Seq=15414141414 Win=65535 Len=0
12.3.0005103442	10.0.1.5	10.0.1.2	TCP	96	56186 → 9999 [ACK] Seq=12141414141 Win=65535 Len=0
13.4.010227410	10.0.1.5	10.0.1.2	TCP	96	56186 → 9999 [PSH, ACK] Seq=12141414141 Win=65535 Len=0
14.4.010723345	10.0.1.2	10.0.1.5	TCP	117	9999 → 56186 [PSH, ACK] Seq=20514141414 Win=65535 Len=0
15.4.010723556	10.0.1.5	10.0.1.2	TCP	96	56186 → 9999 [ACK] Seq=15141414141 Win=65535 Len=0
16.5.012710599	10.0.1.5	10.0.1.2	TCP	97	56186 → 9999 [PSH, ACK] Seq=15141414141 Win=65535 Len=0
17.5.012813694	10.0.1.2	10.0.1.5	TCP	117	9999 → 56186 [PSH, ACK] Seq=25614141414 Win=65535 Len=0

Fig. 8: Forward packets in wireshark

4) *Step 4. Traffic Redirection:* As shown in Fig. 7b, after completing the forwarding tests, `ryu_redirect.py` was executed on controller node *c0*. Following a five-second pause to allow the previous flow entries to expire, the client initiated a new TCP connection to *server1*, while the controller transparently installed redirect rules that steered the traffic to *server2*. Throughout this procedure, Wireshark continuously captured packets, and the trace in Fig. 9 confirms the successful redirection.

Time	Source	Destination	Protocol	Length	Info
1.0.000000000	10.0.1.5	10.0.1.3	TCP	96	46834 → 9999 [PSH, ACK] Seq=1414141414 Win=65535 Len=0
2.0.000115242	10.0.1.3	10.0.1.5	TCP	117	9999 → 46834 [PSH, ACK] Seq=1414141414 Win=65535 Len=0
3.0.000127357	10.0.1.5	10.0.1.3	TCP	96	46834 → 9999 [ACK] Seq=33141414141 Win=65535 Len=0
4.1.000191505	10.0.1.5	10.0.1.3	TCP	96	46834 → 9999 [PSH, ACK] Seq=33141414141 Win=65535 Len=0
5.1.0002010160	10.0.1.3	10.0.1.5	TCP	117	9999 → 46834 [PSH, ACK] Seq=52141414141 Win=65535 Len=0
6.1.0002022552	10.0.1.5	10.0.1.3	TCP	96	46834 → 9999 [ACK] Seq=65141414141 Win=65535 Len=0
7.2.0003094376	10.0.1.5	10.0.1.3	TCP	96	46834 → 9999 [PSH, ACK] Seq=65141414141 Win=65535 Len=0
8.2.004163438	10.0.1.3	10.0.1.5	TCP	117	9999 → 46834 [PSH, ACK] Seq=97141414141 Win=65535 Len=0
9.2.004185347	10.0.1.5	10.0.1.3	TCP	96	46834 → 9999 [ACK] Seq=97141414141 Win=65535 Len=0
10.3.000461595	10.0.1.5	10.0.1.3	TCP	96	46834 → 9999 [PSH, ACK] Seq=97141414141 Win=65535 Len=0
11.3.0005093939	10.0.1.3	10.0.1.5	TCP	117	9999 → 46834 [PSH, ACK] Seq=129141414141 Win=65535 Len=0
12.3.000607334	10.0.1.5	10.0.1.3	TCP	96	46834 → 9999 [ACK] Seq=129141414141 Win=65535 Len=0
13.4.000607213	10.0.1.5	10.0.1.3	TCP	96	46834 → 9999 [PSH, ACK] Seq=129141414141 Win=65535 Len=0
14.4.000743065	10.0.1.3	10.0.1.5	TCP	117	9999 → 46834 [PSH, ACK] Seq=161141414141 Win=65535 Len=0
15.4.000759939	10.0.1.5	10.0.1.3	TCP	96	46834 → 9999 [ACK] Seq=161141414141 Win=65535 Len=0
16.5.010662393	10.0.1.5	10.0.1.3	TCP	96	46834 → 9999 [PSH, ACK] Seq=161141414141 Win=65535 Len=0
17.5.010810031	10.0.1.3	10.0.1.5	TCP	117	9999 → 46834 [PSH, ACK] Seq=256141414141 Win=65535 Len=0

Fig. 9: Redirect packets in wireshark

5) *Step 5. Network Latency Calculation:* We repeated the experiment multiple times and collected packet traces with Wireshark to obtain more reliable measurements of network latency during the TCP three-way handshake.

C. Test Results

The results for the five tasks are reported in the following sections.

1) *Network Topology and Address Verification:* We implemented the required SDN network topology (comprising hosts, a switch, and a controller) by writing `networkTopo.py` using the Mininet Python library. The correctness of the host IP and MAC address configuration was verified, as shown in Fig. 4, and the `pingall` command was used to check for packet loss between nodes, as shown in Fig. 5.

2) *Flow Table Check and Management*: We executed `ryu_forward.py` and `ryu_redirect.py` to install flow table entries, and then issued `sudo ovs-ofctl -O OpenFlow13 dump-flows s1` on the switch to verify that the entries included the parameter `idle_timeout=5` and correctly maintained packet counters and traffic statistics in Fig. 6. After stopping the traffic and waiting for five seconds, the flow table was examined to confirm that all idle entries had been removed.

By using Wireshark to capture the three phases of the TCP handshake for the forwarding and redirection cases, respectively, we analyzed the corresponding handshake latencies.

3) Traffic Forwarding Analysis:

- **Total Delay:** 0.00284 s (averaged over ten trials), indicating a relatively low latency.
- **SYN → SYN+ACK delay:** As shown in Fig. 10a, this component accounts for more than 99% of the handshake delay, indicating that initial end-to-end path setup and control-plane processing dominate the latency. Fig. 10b shows that this delay remains relatively stable.
- **SYN+ACK → ACK delay:** As shown in Fig. 10, this phase contributes only a small and stable fraction of the total handshake delay, indicating that once flow entries and caches are installed, the SYN-ACK and final ACK are forwarded purely in the data plane with negligible additional latency.

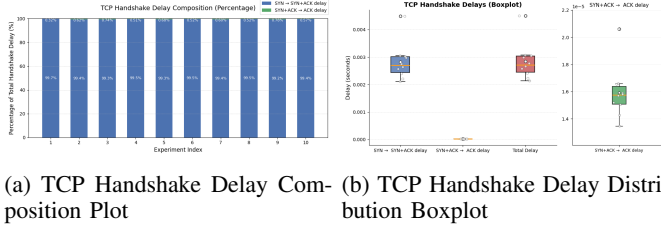


Fig. 10: Presentation of Delays in the Three Phases of the TCP Handshake During the Implementation of Forwarding

4) Traffic Redirection Analysis:

- **Total Delay:** 0.00589 s (averaged over ten trials), indicating a relatively high latency.
- **SYN → SYN+ACK delay:** As shown in Fig. 11a, Similar to the forwarding case, this component accounts for approximately 99% of the total delay, with the dominant latency arising from end-to-end path establishment and the additional complexity of the redirection logic. Fig. 11b shows that this delay exhibits relatively large variability.
- **SYN+ACK → ACK delay:** As shown in Fig. 11, similar to the forwarding case, the delay in this phase is small and remains stable.

5) *Delay Analysis*: As shown in Fig. 10, 11, 12, we observe that the redirection latency is significantly higher than the forwarding latency. Consistent with the previous analysis, this additional delay arises mainly in the SYN-to-SYN+ACK phase, where extra decision logic and the construction of a more complex NAT are required. The figures also show that the redirection latency exhibits markedly greater variability

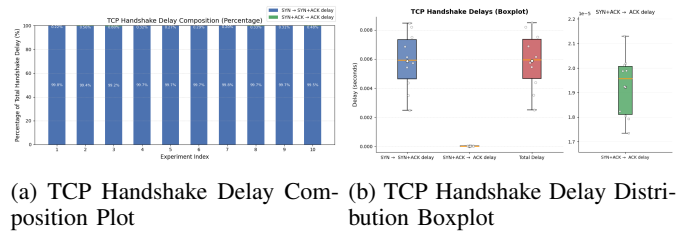


Fig. 11: Presentation of Delays in the Three Phases of the TCP Handshake During the Implementation of Redirecting

than the forwarding latency, again reflecting the higher complexity of the redirection logic.

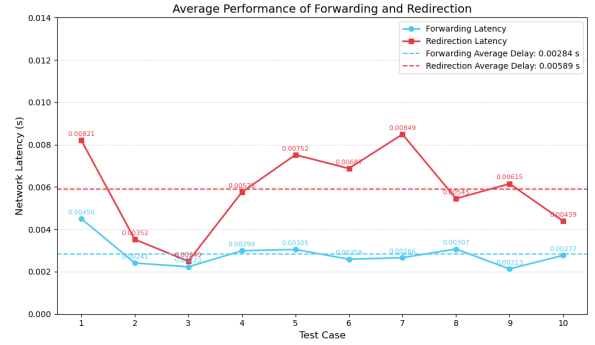


Fig. 12: Forwarding and Redirecting Average Performance

VI. CONCLUSION

In conclusion, we implemented an SDN topology with one client, two servers, and an OpenFlow switch managed by a Ryu controller. The forwarding controller performs MAC learning and installs short-lived flow entries, while the redirection controller rewrites headers to steer traffic transparently. Through systematic testing with Wireshark, both forwarding and redirection processes were validated. The measured latency analysis shows that redirection introduces slightly higher delay due to the additional decision logic and header rewriting.

Future work will optimize flow timeout settings, improve efficiency through asynchronous handling, and explore adaptive load balancing and monitoring functions for larger network scenarios.

REFERENCES

- [1] R. Botez, A. G. Pasca, A. T. Sferle, I. A. Ivanciu, and V. Dobrota, "Efficient network slicing with SDN and heuristic algorithm for low latency services in 5G/B5G networks," *Sensors*, vol. 23, no. 13, p. 6053, Jul. 2023.
- [2] M. F. Hyder, T. Fatima, S. M. Khan, and S. Arshad, "Countering crossfire DDoS attacks through moving target defense in SDN networks using OpenFlow traffic modification," *Transactions on Emerging Telecommunications Technologies*, vol. 34, no. 12, pp. 1–22, Dec. 2023.
- [3] Y. Wang, J. Bi, and K. Zhang, "A tool for tracing network data plane via SDN/OpenFlow," *Science China Information Sciences*, vol. 60, no. 2, pp. 70–82, 2017.